

Kriptografija i računarska zaštita – Usmeni ispit

1. Ako imamo bitmapu $M \times N$ dimenzija koliku poruku možemo ubaciti u takvu sliku primjenom LSB?

Ako je bitmapa dimenzija $M \times N$ znači da ima ukupno $M \times N$ piksela, kako se svaki piksel sastoji od 3 komponente (R – crvena, G – zelena, B – plava), tj od 3 bajta, dolazimo do toga da imamo ukupno $3 \times M \times N$ bajta.

Kako tehnika LSB predstavlja „ubacivanje“ informacija na posljednji bit u bajtu, tj bit najmanje težine, znači da u ovakvu bitmapu možemo ubaciti poruku od $3 \times M \times N$ bita.

Npr: Ako imamo bitmapu 100×100 , imamo ukupno $100 \times 100 = 10000$ piksela, odnosno 30000 bajtova, to nam znači da možemo ubaciti poruku od 30000 bita.

a. Koliko se bajtova promjeni u prosjeku za prethodno pitanje?

U prosjeku se promjeni 50% bajtova.

2. Veličine blokova i ključeva kod RC4, DES i AES?

- **RC4**
 - ključ promjenljive dužine od 1 do 256 bajta (8 – 2048 bita)
 - nema blokova, jer je **stream cipher (strim šifarski)** algoritam
- **DES**
 - dužina ključa (inicijalno 64, ali efektivno) 56 bita
 - broj rundi 16
 - dužina bloka 64 bita
 - **3DES**
 - dužina ključa 168
 - dužina bloka 64
 - broj rundi 48
- **AES**
 - dužina ključa 128/192/256 bita
 - u zavisnosti od dužine ključa imamo 10, 12 ili 14 rundi
 - dužina bloka 128 bita

3. Šta je konfuzija i difuzija?

- **Konfuzija** – cilj da se veza između ključa i kriptovanog teksta učini što je moguće kompleksnijom. Konfuzija čini izlaz zavisnim od ključa – idealno bi bilo da jedan bit ključa utiče na sve bite izlaza. Konfuzija otežava pronalazak ključa iz kriptovanog teksta.
 - konfuzija se prije (u istorijskim algoritmima) radila supstitucijom, a danas se radi pomoću s-box-ova (takođe supstitucija)
 - shift cipher – osnovna operacija je supstitucija
 - s-box-ovi u modernim algoritmima (supstitucija)

- **Difuzija** – preraspoređivanje bita ulazne poruke na takav način da se svaka redundantnost u otvorenom tekstu raširi preko kriptovanog teksta. Difuzija čini izlaz zavisnim od prethodnih ulaza (otvoreni ili plain text) – idealno bi bilo da svaki bit ulaza utiče na sve bite izlaza. Difuzija otežava pronalazak otvorenog teksta iz kriptovanog teksta.
 - (nekada) transpozicija – mijenjanje pozicije slova u otvorenom tekstu
 - (sada) p-box-ovi u modernim algoritmima (permutacija)

Kombinacija konfuzije i difuzije je dobra praksa za dobijanje dobre šeme (dobri primjeri su DES i AES). Simetrični algoritmi **moraju biti kompleksni** (kako ne bi mogli biti jednostavno analizirani) i **moraju biti jendostavni** (kako bi mogli biti lako implementirani). Zato moderni algoritmi koriste „mješavinu“ konfuzije i difuzije, i to kroz nekoliko rundi, kako bi postigli potrebnu kompleksnost uz dodatnu (ali malu) cijenu nešto kompleksnije implementacije.

Efekat lavine – funkcija ima dobar efekat lavine, ako promjena jednog bita ulaza rezultira promjenom minimalno polovine bita izlaza.

Efekat kompletnosti – svaki izlazni bit je kompleksna funkcija svih ulaznih bita.

Dobro dizajniran algoritam je onaj koji ispunjava:

1. efekat lavine
2. efekat kompletnosti
3. nepredvidljivost – slučajnost (*npr: za funkciju koja ekspanduje ključ na potreban broj bita ona mora da bude baš nepredvidiva – tj. ponavljanje (dupliranje) istog ključa nije nepredvidivo*)

Loše dizajniran algoritam – ima suviše mnogo predvidljivosti. Mnogi algoritmi su razbijeni. Preporuka je da se ne razvijaju sopstveni algoritmi (*generalno se može desiti da mi razvijemo algoritam koji je teorijski jako dobar, ali da ga implementiramo loše pa ga neko razbije zbog naših bagova*), već da se koriste dokazano sigurni algoritmi (ne postoje dokazano sigurni algoritmi, već sigurnim algoritmima smatramo one za koje nije dokazano da su razbijeni, **ondnosno** nisu nesigurni).

4. No integrity (bit-flipping) napad?

No integrity predstavlja jedan od napada na stream šifre, pa ćemo obraditi oba napada, a ne samo no integrity.

- **Two time pad** – napad u kom dvije različite poruke kriptujemo istim ključem, tj isti ključ ubacujemo u PRNG i dobijamo isti strim ključ kojim ćemo šifrovati dvije poruke. Napadač na primjer uhvati ta dva šifrata kriptovana istim strim ključem i njihovim XOR-ovanjem dobija **m1 XOR m2** (jer je $c1 = m1 \text{ XOR } \text{PRNG}(k)$ i $c2 = m2 \text{ XOR } \text{PRNG}(k)$). Kako sada napadač ima $m1 \text{ XOR } m2$, na osnovu dovoljne redundantnosti u engleskom jeziku relativno lako doazi do **m1** i **m2** (*još ako je jedna od poruka početak iz SMTP protokola MAIL FROM:; odmah znamo i početak druge i tako dalje do cijelih poruka*)
- **No integrity (bit-flipping) napad** – napadač uhvati šifrat i dodatno ga XOR-uje neki svojim ključem, to izgleda ovako:
 - korisnik kriptuje poruku m sa ključem k ($c = m \text{ XOR } k$)

- napadač uhvati šifrat c i dodatno ga kriptuje sa svojim ključem p ($c \text{ XOR } p = m \text{ XOR } k \text{ XOR } p$)
- primalac dobija $m \text{ XOR } k \text{ XOR } p$ i kada ga dešifruje sa $m \text{ XOR } k \text{ XOR } p \text{ XOR } k$ dobija $m \text{ XOR } p$, odnosno dobija ono što je napadač htio da primalac dobije – izmjenjenu poruku dodatni XOR-ovanjem
- **primjer no integrity napada** (opet MAIL FROM: (samo sada) BOB):
 - Normalan scenario
 - riječ BOB je kriptovana sa $BOB \text{ XOR } KEY$ i na prijemnoj strani bito sa dekrijpcijom $BOB \text{ XOR } KEY \text{ XOR } KEY$ dalo BOB
 - Scenario napada – napadač zna na kojoj poziciji se nalazi riječ BOB
 - želi da umjesto BOB piše EVE i pravi $BOB \text{ XOR } EVE$
 - tu vrijednost XOR-uje sa onim što se nalazi u šifratu (nalazi se $BOB \text{ XOR } KEY$)
 - zapravo dobija $BOB \text{ XOR } EVE \text{ XOR } BOB \text{ XOR } KEY$
 - $BOB \text{ XOR } BOB$ se poništava i ostaje samo $EVE \text{ XOR } KEY$
 - primalac sada kada dešifruje – primjeni $EVE \text{ XOR } KEY \text{ XOR } KEY$ dobija poruku EVE, a ne BOB kako je pošiljalac poslao
 - ZAKLJUČAK: Napadač je podmetnuo lažnu informaciju, a da nije ni znao koji je ključ korišten.
 - Napadač može prevariti i tako što će u REPLY TO polje u SMTP protokolu da se odgovori automatski šalju njemu i mi kao korisnik koji odgovara na mejl ako ne pogledamo dodatno (ručno) na koju mejl adresu šaljemo odgovor, poslaćemo napadaču.

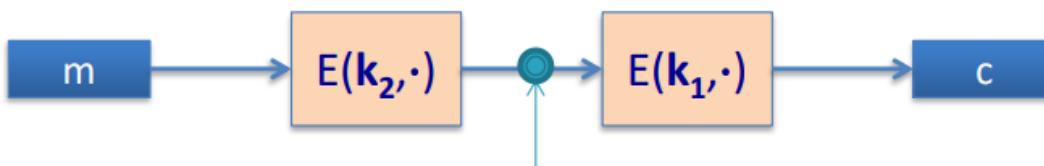
5. Double DES složenost i napad?

Po samoj priči je bolji od 3DES, jer je brži, a dužina ključa mu je $2 \cdot 56 = 112$ ($2 \cdot \text{DES}$) i to nam je dovoljno, jer današnji standard kaže da je 112 dovoljno (minimalno).

Ne postoji zbog meet-in-the-middle ranjivosti.

Da bi se izveo napad meet-in-the-middle napadač mora imati i poruku (plain text) i šifrat. Napadač se trudi da nađe ključeve oba DES-a, to radi tako što otvorenu poruku kriptuje sa svih 2^{56} mogućih ključeva u prvom DES-u, a šifrat dekriptuje drugim DES-om dok ne dobije x koje odgovara nekom od šifrata prvog DES-a, kada to dobije pronašao je i k_1 i k_2 . Ovim napadom se generiše 2^{56} šifrovanja u prvom DES-u i 2^{56} dešifrovanja u drugom DES-u što je ukupno 2^{57} što nam govori da je 57 bita efektivna dužinu ključa Double DES algoritma.

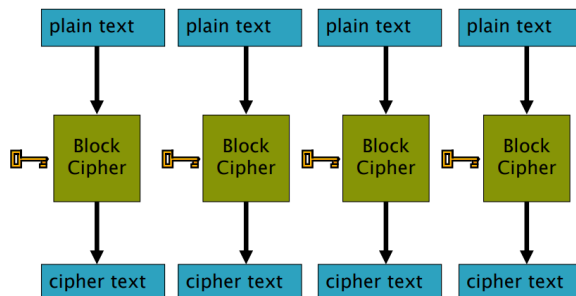
Konačna kompleksnost Double DES algoritma je $2^{56} \log(2^{56}) + 2^{56} \log(2^{56}) < 2^{63}$ (prvi sabirak predstavlja kreiranje i sortiranje tabele, a drugi je pretraga tabele).



6. Modovi rada kod simetričnih algoritama, koji je najbrži i koji je najbolji?

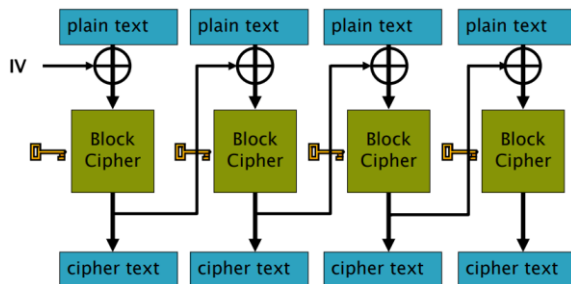
Modovi rada kod simetričnih algoritama su sljedeći:

- **ECB – Electronic codebook**



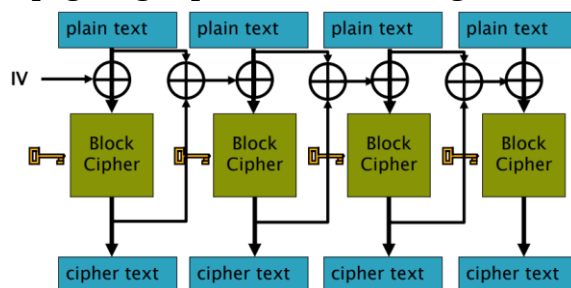
- Veoma efikasan i brz, jer se svaki blok plain text poruke šifrue nezavisno od ostalih blokova i dobija se blok šifrata (šifrovane poruke).
- Na efikasnost utiče i to što je i proces enkripcije i proces dekripcije moguće paralelizovati.
- Problem ECB moda rada je taj što, ako na ulazu imamo iste blokove, dobićemo iste blokove šifrata na izlazu, jer se u svakom bloku koristi isti ključ
- Često se koristi u komercijane svrhe.
- *Možda najveći nedostatak ECB moda možemo vidjeti pri enkripciji slika, jer one imaju veliki broj istih blokova koji će onda biti kriptovani na isti način i kriptovanjem cijeli slike dobijamo opet prilično čitljiv tekst na slici (primjer sa slikom „Google“ logo u kom dobijamo slike samo sa drugim bojama, ali uvijek možemo pročitati da tu stvarno piše „Google“).*

- **CBC – Cipher-block chaining**

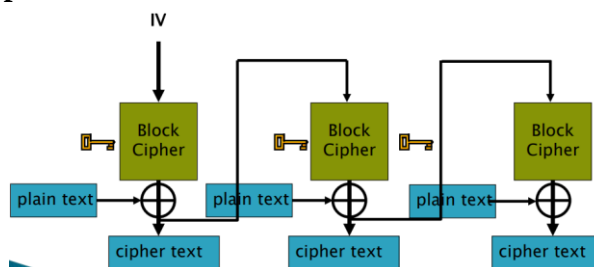


- U prvoj iteraciji se plain text prvog bloka XOR-uje sa inicijalizacionim vektorom (IV) i takav ulazi u funkciju za šifrovanje koja koristi naš ključ i dobija se blok šifrata.
- U svakoj sljedećoj rundi se šifrat iz prethodne runde XOR-uje sa plain text-om te runde i taj rezultat XOR-ovanja ulazi u funkciju za šifrovanje, nakon čega dobijamo šifrat i proces se nastavlja
- U CBC modu rada enkripcija ne može biti paralelizovana (mora biti sekvencijalna), jer svaki sljedeći blok koji se enkriptuje zavisi od šifrata prethodnog bloka

- to nam govori da je ovaj mod (CBC) manje efikasniji od ECB, ali je sigurniji (jer svaki blok zavisi od šifrata prethodnog bloka, pa samim tim ako na ulazu imamo više istih blokova, nećemo dobiti iste šifrate, jer se prije enkripcije plain text XOR-uje sa prethodnim šifratom)
- Za razliku od enkripcije, dekripcija u CBC modu rada može biti paralelizovana, jer u trenutku dekripcije poznajemo sve šifrate, a za dekripciju jednog šifrata nam je potreban konkretan taj šifrat i šifrat prethodnog bloka (nakon dekripcije konkretnog bloka šifrata dobijena vrijednost se XOR-uje sa šifratom prethodnog bloka i dobijamo plain text konkretnog bloka)
- Veoma često korišten mod rada.
- **PCBC – Propagating Cipher-block chaining**



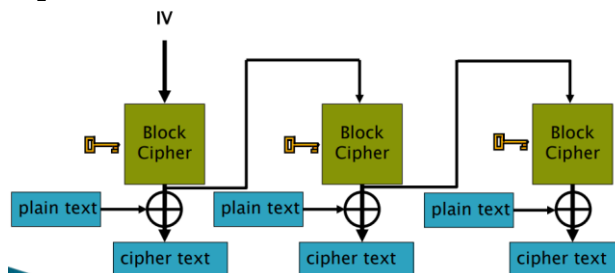
- Sličan način rada CBC modu uz jednu izmjenu u radu.
- U prvoj iteraciji se plain text prvog bloka XOR-uje sa inicijalizacionim vektorom (IV) i takav ulazi u funkciju za šifrovanje koja koristi naš ključ i dobija se blok šifrata.
- Za razliku od CBC moda u PCBC modu se izlaz svakog bloka (šifrat) XOR-uje sa plain text-om tog istog bloka, pa se dobijeni rezultat tog XOR-ovanja XOR-uje sa plain tekstom sljedećeg bloka i taj rezultat ulazi u funkciju za šifrovanje, nakon čega dobijamo šifrat bloka.
- U ovom modu rada su i enkripcija i dekripcija sekvencijalne, jer svaki sljedeći blok zavisi i od šifrata i od plain text-a prethodnog bloka.
- Kao i CBC i PCBC je često korišten.
- **CFB – Cipher feedback**



- U prvom bloku inicijalizacioni vektor (IV) ulazi u funkciju za šifrovanje, nakon izlaza iz funkcije taj rezultat XOR-ujemo sa plain text-om tog bloka i dobijamo šifrat tog bloka.

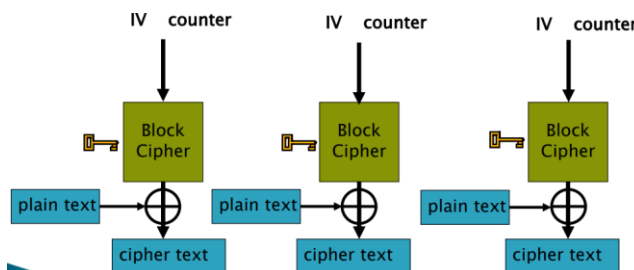
- U svakom sljedećem bloku ulaz funkcije je šifrat prethodnog bloka, nakon čega se izlaz funkcije XOR-uje sa plain text-om tog bloka i dobijamo šifrat tog bloka.
- Proces enkripcije je sekvencijalan (jer svaki sljedeći blok koji se enkriptuje zavisi od šifrata prethodnog bloka), a dekripcija se može paralelizovati.

- **OFB – Output feedback**



- Veoma sličan CFB modu, samo što je ulaz u funkciju sljedeće runde izlaz funkcije prethodne runde, a ne šifrat prethodne runde.
- Ne postoji mogućnost paralelizacije ni enkripcije, ni dekripcije.

- **CTR – Counter mode**



- Prvo se bira početna vrijednosti inicijalizacionog vektora (IV).
- Ulaz u svaki blok za šifrovanje je IV + broj bloka.
- Nakon obrade funkcije dobijene rezultat XOR-ujemo samo plain text-om tog bloka i dobijamo šifrat tog bloka.
- Moguće je paralelizovati i enkripciju i dekripciju.
- Mora se dobro paziti da se nikada ne koristi isti IV za isti ključ.

7. NIST preporuke za dužine ključa?

Preporuke za bitske dužine **simetričnih** algoritama:

- za period 2007-2010. godine – minimalna bitska dužina ključa simetričnog algoritma iznosi 80
- za period 2011-2030. godine – minimalna bitska dužina ključa simetričnog algoritma treba da iznosi 112
- Prema radu „Minimal Key Lengths for Symmetric Ciphers“ (autori Blaze, Diffie, Rivest, Schneier, Shimomura, Thompson, Weiner) iz 1996. godine:
 - nesigurnim se smatra korištenje ključeva simetričnih algoritama čija je bitska dužina manja ili jednaka 64

Preporuke NIST-a za kombinaciju simetrični-asimetrični:

period	algoritam - dužina ključa	
	simetrični	asimetrični
	80	1024
2020 - 2030	112	2048
2019 - 2030+	128	3072
2019 - 2030+	192	7680
2019 - 2030+	256	15360

8. Diffie Hellman i PFS (Perfect Forward Secrecy)?

Diffie Hellman je jedan od prvih protokola koji je razvijen za razmjenu javnih ključeva, tj. key agreement, a ne key exchange protokol. Dvije strane se dogovaraju oko uspostave tajnog ključa, tj. razmjenjuju se javni dijelovi ključa i nakon razmjene se kombinuju sa privatnim i dobija se dijeljeni ključ.

Obe strane znaju g i n (veliki prosti brojevi, n je reda veličine 2000-4000 bita, nema potrebe da bude veći, jer se ne dobija na sigurnosti, a gubi se na performansama, g je prost broj koji je manji od n). Korisnik A ima privatni ključ a (veliki broj, slučajna vrijednost manja od n), korisnik B ima privatni ključ b (veliki broj, slučajna vrijednost manja od n). Cilj algoritma je da A šalje ka B $g^a \bmod n$, a B šalje ka A $g^b \bmod n$, da bi na kraju oba korisnika mogla izračunati $g^{ab} \bmod n$.

Napadi na Diffie Hellman:

1. Pasivni napadač može samo da uhvati $g^a \bmod n$ i $g^b \bmod n$ i može to dvoje da pomnoži, tj. da dobije $g^{a+b} \bmod n$, ali ne može doći do $g^{ab} \bmod n$. Takođe ne može izračunati a ili b za dato g i n (još jednom da se napomene g i n su javno dostupni), dok je $g^a \bmod n$ i $g^b \bmod n$ lako izračunati.
2. Aktivni napadač M koji može da manipuliše porukama i ima svoj privatni ključ m može da:
 - 2.1. uhvati $g^a \bmod n$ i korisniku A vraća $g^m \bmod n$ (korisnik A misli da je dobio odgovor od korisnika B, tj. da je dobio $g^b \bmod n$) i sada korisnik A računa $g^{am} \bmod n$ i napadač isto računa $g^{am} \bmod n$, tj. korisnik A i M su uspostavili povjerenje (imaju međusobni dijeljeni ključ).
 - 2.2. korisniku B pošalje $g^m \bmod n$ (B misli da je dobio $g^a \bmod n$ od korisnika A), B vraća napadaču $g^b \bmod n$ i sada B i M računaju $g^{bm} \bmod n$ i uspostavili su povjerenje (imaju međusobni dijeljeni ključ).
 - 2.3. sada kada korisnik A šalje poruku korisniku B, ona zapravo ide ka korisniku M, M može da je čita, mijenja, ... i nakon toga proslijedi „lažnu“ poruku ka B (a da B misli da je to sve stiglo od korisnika A) (može i sve obrnuto da B šalje ka A) – napadač je izveo MITM (Men-In-The-Middle) napad.

Postavlja se pitanje „Kako onda riješiti ovaj problem kod Diffie Hellman-a?“. Rješenje je da korisnici A i B prethodno na neki siguran način imaju razmijenjene javne ključeve, recimo RSA algoritma. Onda bi A kada generiše $g^a \bmod n$ to digitalno potpisao svojim privatnim ključem i slao preko mreže. Napadač i sada može ovu poruku presresti, ali mu ništa ne vrijedi kada nema privatni ključ korisnika A da bi potpisao $g^m \bmod n$ i zavarao korisnika B.

Sada se postavlja pitanje „Što će nam Diffie Hellman, ako već imamo na siguran način razmijenjene RSA ključeve?“. Odgovor leži u **Perfect Forward Secrecy**. Ovo je slučaj u kome je došlo do kompromitacije privatnog ključa RSA algoritma.

Ako se koristila digitlna envelope, kada napadač dođe do privatnog RSA ključa, on može da dekriptuje sve prethodne envelope (jer je za svaku korišten isti ključ). Dekrijpcijom svih envelope može da dođe do svih sesijskih ključeva simetričnih algoritama i ukoliko je skladištio prethodno razmijenjene šifrate može da ih dekriptuje (u nekom prosječnom slučaju moći će da dekriptuje polovinu poruka, ako su korisnici ravnomjerno razmjenjivali, tj. šifrovali poruke). Isto tako napadač može da dekriptuje i sve sljedeće šifrate koji se budu razmjenjivali.

Zbog ovoga se koristi Diffie Hellman i napadač kompromitacijom privatnog RSA ključa ne može da dođe do plain text poruka koje su razmijenjene prije kompromitacije, čak i ako je skladištio šifrate. Ovo nije moguće, jer su brojevi a i b koji se koriste kao ključevi slučajno generisane vrijednosti koje se generišu za svaku novu poruku. Buduće poruke se opet hoće moći dekriptovati, jer će se napadač praviti da je korisnik čiji je privatni ključ kompromitovao i moći će potpisati $g^m \bmod n$.

9. Eliptičke krive?

Eliptička kriva je funkcija oblika:

$$y^2 = x^3 + ax + b$$

gdje su x i y horizontalna i vertikalna koordinata.

Kriptografija eliptičkih krivih je bazirana na jedinstvenim osobinama eliptičkih krivih:

- horizontalna simetrija
- presjek sa pravom u tri tačke
- presjek sa tangentom u dvije tačke

Uzimajući u obzir ove osobine, moguće je definisati tri operacije sa tačkama ECC:

- **Sabiranje tačaka:** $C = A + B$
 - sabiranje dvije tačke se vrši određivanjem tačke presjeka eliptičke krive (ECC) i prave određene datim tačkama i njenom refleksijom oko x-ose (zbir dvije tačke je takođe tačka koja pripada toj krivoj).
- **Dupliranje tačaka:** $C = 2A$
 - dupliranje tačke, odnosno sabiranje dvije iste tačke, vrši se određivanjem tačke presjeka ECC i tangente na ECC u tački koja se duplira, te njenom refleksijom oko x-ose.
- **Skalarno množenje tačaka:** $C = kA$
 - množenje tačaka se realizuje kao kombinacija sabiranja i dupliranja tačaka (određeno je *double-and-add* algoritmom), množenje se definiše kao sabiranje tačke same sa sobom k puta.

Korištenjem skalarnog množenja tačke, moguće je pomnožiti neku početnu tačku P na krivoj vrlo velikim slučajnim brojem d

- d predstavlja privatni ključ
- rezultat množenja Q je javni ključ u ECC sistemu: $Q = dP$

Računski je nemoguće ovu operaciju izvršiti unazad – što formira jednosmjernu funkciju koja se naziva **diskretni logaritamski problem eliptičke krive**. Odeđivanje javnog ključa na osnovu privatnog ključa i početne tačke je vrlo jednostavno, ali se određivanje privatnog ključa uz poznavanje javnog ključa i početne tačke P **samtra** izuzetno zahtjevnom operacijom. Potencijalni pokušaj da se odredi privatni ključ bi mogao biti sabiranje početne tačke same sa sobom i brojanje operacija sabiranja sve dok se ne dobije javni ključ, ali s obzirom na to da se radi o vrlo velikom broju operacija, ovo je prekično neizvodljivo.

Praktična implementacija ECC-a koristi eliptičku krivu nad konačnim poljem, jer se može desiti da za neko x y ide u beskonačnost, ograničenje se postiže dodavanje modula.

Kriptografija eliptičkih krivih pruža istu sigurnost kao i RSA uz značajno manju dužinu ključa. Ključevi manje dužine zahtjevaju manji propusni opseg i manje memorije, što je izuzetno značajno za IoT uređaje, pametne telefone, ...

Simetrični algoritmi (dužina ključa u bitima)	RSA i Diffie-Hellman (veličina modula u bitima)	ECC (dužina ključa u bitima)
56	512	112
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

Postoje i verzije ECDSA (DSA pomoću eliptičkih krivih) ili ECDH (Diffie Hellman – postoji zaštita od pasivnih napadača, ali ne i od aktivnih zbog MITM napada).

10. Digitalni potpis?

Digitalni potpis predstavlja elektronsku analogiju stvarnog potpisa rukom. Slično potpisu rukom digitalni potpis mora biti otporan na falsifikovanje, primalac mora biti u stanju da ga verifikuje i pošiljalac, tj. onaj koji potpisuje ne smije biti u stanju da kasnije porekne to što je potpisao (obezbeđuje se neporecivost).

Glavna razlika u odnosu na potpis rukom je to što digitalni potpis mora biti funkcija čitavog dokumenta za koji se kreira, kako se ne bi mogao kopirati i dodati na bilo koji dokument. U slučaju da je konstanta veoma lako bi se mogao kopirati i dodavati na dokumente (slično kao skeniranje ručnog potpisa i **prinatanje** na dokumente).

Digitalnim potpisom se obezbeđuje integritet, autentičnost i neporecivost. Predstavlja postupak kojim se određeni blok podataka ili standardizovane poruke kriptografski obilježava privatnim ključem pošilajoca. Digitalno potpisan dokument, ako nije mijenjan, može se iskoristiti x puta, ne možemo da razlikujemo kopiju od originala, ali to i nije toliko bitno, bitno da dokument nije mijenjan.

Osnovne karakteristike digitalnog potpisa:

- **autentičnost** – digitalni potpis je garancija primaocu da je određeni pošiljalac poslao poruku

- **nemogućnost falsifikovanja** – digitalni potpis je dokaz da je pošiljalac, i niko drugi, potpisao dokument
- **nije ponovo upotrebljiv** – digitalni potpis je dio dokumenta, tako da maliciozni učesnik ne može potpis upotrebiti na drugom dokumentu
- **očuvanje integriteta** – digitalno potpisani dokument se ne može promijeniti, a da se ta promjena ne detektuje
- **neporecivost** – pošiljalac koji je poslao dokument ne može kasnije poreći da je to učinio

Dokumenti se mogu digitalno potpisivati na različite načine:

- **pomoću simetričnih šifarskih sistema i arbitratora**
 - Arbitrator je treća strana kojoj vjeruju svi učesnici u komunikaciji, on ima na neki siguran način razmijenjene tajne ključeve sa svakim korisnikom.
 - Ako korisnik A želi da šalje poruku korisniku B on kriptuje poruku ključem koji je razmijenio sa arbitratorom (ne može salti direktno ka B, jer nema ključ B korisnika).
 - Arbitrator dekriptuje poruku koju je dobio od A i kriptuje je ključem korisnika B zajedno sa tvrdnjom da poruka stiže od A.
 - B dekriptuje svojim ključem i vidi da je poruka stigla od A.
 - Problem ovog pristupa je što arbitrator čita sve poruke i to što arbitrator predstavlja usko grlo, jer sve poruke moraju ići preko njega.
 - Ovaj način takođe nije otporan na DOS napade.
- **pomoću asimetričnih šifarskih sistema**
 - Poruka se kriptuje privatnim ključem pošiljaoca i šalje se primaocu, primalac kada dobije poruku dekriptuje je javim ključem pošiljaoca i utvrdi da je stvarno on poslao.
 - Ovo se rijetko radi, jer se veliki dokumenti sporo kriptuju asimetričnim algoritmima.
- **pomoću asimetričnih šifarskih sistema i jednosmjernih hash funkcija**
 - Pošiljalac kreira hash dokumenta i onda taj hash enkriptuje svojim privatnim ključem, tj digitalno ga potpiše.
 - Digitalni potpis zajedno sa dokumentom šalje primaocu, dokument je čitljiv, neizmijenjen.
 - Primalac pravi otisak (hash) tog dokumenta, javnim ključem pošiljaoca dekriptuje digitalni potpis i provjerava da li se dva hash-a poklapaju, ako se poklapaju utvrđen je integritet (nije tajnost).
 - *Ovdje može da postoji problem ukoliko nađemo dva dokumenta koji imaju isti hash, odnosno nađemo koliziju. Velika vjerovatnoća da takav dokument neće umati upotrebnu vrijednost, jer npr. ako pravi dokument predstavlja transakciju novca sa računa A na račun B, dokument sa istim hash-om će gotovo sigurno biti neki dokument koji nema veze sa transakcijom novca, tako da će se taj dokument samo odmaciti i neće biti većih problema.*

Asimetrični algoritam koji se koristi samo za digitalno potpisivanje je DSA.

a. Šta ako želimo i digitalni potpis i enkripciju?

Prvo kreiramo digitalni potpis na prethodni način, hash dokumenta kriptujemo privatnim ključem, zatim generišemo ključ za simetrični algoritam i kriptujemo dokument zajedno sa digitalnim potpisom pomoću simetričnog algoritma, a ključ simetričnog algoritma pakujemo u digitalnu envelopu i kriptujemo je javnim ključem primaoca. Sada možemo primaocu poslati i kriptovani dokument sa digitalnim potpisom i digitalnu envelopu. Na ovaj način smo obezbijedili **tajnost, integritet, autentičnost i neporecivost**.

Postoji i opcija da prvo kriptujemo dokument pa ga onda potpišemo, ali to nije ispravno. Istina je da smo mi na taj način postigli sve što želimo, ali nije to to. U tom slučaju pošiljalac može tvrditi u nekom trenutku da nije znao šta je potpisao. To bi bilo isto kao da dokument ubacimo u kovertu, a potpišemo se na kovertu, a ne na dokument.

11. Kako vjerovati CA sertifikatu i kako znati koji algoritam koristiti za dekriptovanje?

(Gemini:) Povjerenje u CA (Certification Authority) sertifikat se ne zasniva na nekom posebnom tehničkom algoritmu, već na uspostavljenom povjerenju u samog CA. Evo ključnih aspekata kako se to povjerenje stiče i održava:

1. **Reputacija i kredibilitet CA:** Pouzdani CA su organizacije sa dugogodišnjom reputacijom u industriji. Oni prolaze kroz stroge audite i akreditacije od strane nezavisnih tijela kako bi dokazali da poštuju najbolje prakse u izdavanju i upravljanju sertifikatima. Poznati CA uključuju Let's Encrypt, DigiCert, GlobalSign, Sectigo i mnoge druge.
2. **Uključivanje root sertifikata u pouzdane prodavnice (Trust Store-ove):** Operativni sistemi (Windows, macOS, Linux), veb pretraživači (Chrome, Firefox, Safari, Edge) i druge aplikacije imaju ugrađene liste pouzdanih root CA sertifikata. Ovi root sertifikati su polazna tačka povjerenja. Kada vaš sistem ili pretraživač naiđe na sertifikat koji je izdat od strane CA čiji je root sertifikat prisutan u njegovom trust store-u, on implicitno vjeruje tom CA.
3. **Verifikacija lanca povjerenja (Certificate Chain Validation):** Kao što smo ranije diskutovali, kada se verifikuje neki krajnji sertifikat (npr. veb sajta), proces uključuje izgradnju lanca povjerenja unazad do root CA. Ako se svaki sertifikat u lancu potpiše validnim potpisom svog izdavača, a root sertifikat je u trust store-u, onda se i krajnji sertifikat smatra pouzdanim.
4. **Ograničenja i opoziv sertifikata:** CA mogu ograničiti svrhu i validnost sertifikata koje izdaju. Takođe, imaju procedure za opoziv sertifikata u slučaju kompromitacije ili drugih problema. Informacije o opozvanim sertifikatima se distribuiraju putem CRL (Certificate Revocation List) ili OCSP (Online Certificate Status Protocol).

Informacije o korištenim algoritmima (za potpisivanje i za javni ključ) mogu se vidjeti u detaljima samog digitalnog sertifikata. Kada pregledate sertifikat u veb pretraživaču ili alatu za upravljanje sertifikatima, obično postoji sekcija "Detalji" ili "Tehnički detalji" koja prikazuje:

- Algoritam potpisa (Signature Algorithm): Označava algoritam koji je CA koristio da potpiše ovaj sertifikat.
- Algoritam javnog ključa (Public Key Algorithm): Označava tip algoritma i dužinu ključa koji se koristi u samom sertifikatu.

(ChatGPT:) CA sertifikatu vjerujemo tako što mu vjeruje OS ili web browser. Ovo se dešava kroz sljedeće korake:

- **Root sertifikati:** Operativni sistemi (npr. Windows, Linux, ...) i web pretraživači (Chrome, Firefox, ...) imaju listu **pouzdanih root CA sertifikata**. Ako sertifikat sajta potiče iz te hijerarhije (tj. izdalo ga je neko CA tijelo koje ta lista priznaje), onda mu se vjeruje.
- **Lanac povjerenja (chain of trust):** Kada posjetimo sajt putem HTTPS protokola, on šalje svoj **sertifikat** i često sertifikate **posredničkih CA (intermediate CAs)**. Pretraživač provjerava da li je taj lanac validan i vodi do root CA sertifikata koji je u listi pouzdanih.
- **Provjera potpisa:** Sertifikat sadrži digitalni potpis izdat od strane CA. Pretraživač koristi javni ključ da provjeri potpis. Ako je validan – sertifikatu se vjeruje.

Algoritam koji koristi za dekriptovanje se provjerava iz polja sertifikata:

- **Identifikator algoritma za potpisivanje (eng: Signature Algorithm)** (npr: sha256WithRSAEncryption) – polje koje nam govori o kombinaciji hash funkcije i asimetričnog algoritma koji su korišteni za digitalno potpisivanje sertifikata.
- **Informacije o algoritmu javnog ključa (eng: Public Key Algorithm)** (npr: rsaEncryption) – identifikator algoritma
 - ovo polje je podpolje unutar polja **Informacije o javnom ključu subjekta (eng: Subject Public Key Info)**

12. Šta sve treba provjeriti kod sertifikata?

Validacija digitalnog sertifikata:

- verifikacija da li je digitalni sertifikat izdat od strane CA kojem se vjeruje (mi ne moramo vjerovati svim sertifikatima i obično ne vjerujemo) – ovo postizemo provjerom digitalnog potpisa
- verifikacija perioda validnosti – provjera da li je sertifikat istekao (jer se svaki sertifikat izdaje na neki vremenski period u zavisnosti od tipa sertifikata: korisnički – do 5 godina, root CA – 20, 30, 40 godina)
- verifikacija da digitalni sertifikat nije povučen – da se ne nalazi na CRL listi, sertifikat može i dalje da vrijedi na osnovu polja *Validity: Not After*, ali se može desiti da je povučen iz upotrebe iz nekog razloga (npr: kompromitacija ključa) i takav sertifikat se stvarno ne bi smio više koristiti, iako mu datum važenja nije još istekao
- verifikacija da se digitalni sertifikat koristi prema definisanoj politici
 - Basic Constraints – ekstenzija koja govori da li se radi o sertifikatu CA tijela ili ne (kao i o maksimalnoj „dubini“ putanje koja uključuje i tekući sertifikat)

- Key Usage – ekstenzija koja ograničava upotrebu ključa u tačno određene svrhe, ako postoji ova ekstenzija mora da sadrži barem jednu upotrebu ključa, a kod CA tijela moraju biti upotrebe ključa potpisivanje CRL liste i digitalnog sertifikata
- dodatno za web sajtove: da je naziv domena odgovarajući – provjeravamo CN polje da li se poklapa sa domenom sajta, ako se ne poklapa onda provjeravamo alternativna imena, ako nema ni tu konkretnog naziva onda provjeravamo da li je iskorišten wildcard npr: *.google.com (wildcard se može naći samo kao poddomen, ne smije nikako kao naddomen)

13. Razlika između CRL i OCSP, šta je bolje i kako koji funkcioniše?

CRL (Certificate Revocation List) – lista opozvanih sertifikata predstavlja fajl u kom se nalazi lista serijskih brojeva sertifikata koji su povučeni (dovoljan nam je serijski broj, jer je on jedinstven). CRL listu izdaje CA tijelo i potpisuje je zbog zaštite integriteta i autentičnosti te CRL liste. CRL lista može biti potpisana istim ključem kojim se izdaju sertifikati ili nekim drugim ključem.

OCSP (Online Certificate Status Protocol) je protokol kojim dobijamo status sertifikata za kojeg pitamo, status može biti: dobar, povučen ili nepoznat.

Razlika između CRL liste i OCSP protokola je ta što je OCSP protokol u kom korisnik šalje zahtjev u kom pita da li je neki sertifikat povučen i u odgovoru dobija jedan od tri statusa (poželjno dobar ili povučen). Taj odgovor mora biti potpisan da bi mu korisnik vjerovao, jer se prenosi preko mreže. A CRL lista je lista sa povučenim zahtjevima, pa onda korisnik mora gledati da li se sertifikat od interesa nalazi na listi (povučen je) ili ne (nije povučen).

CRL lista sadrži: **naziv izdavača CRL liste, vrijeme kada je objavljena, vrijeme kada će biti objavljena sljedeća CRL lista** i ovo vrijeme se može prekršiti u smislu da se CRL lista objavi ranije, ali nikako ne smije biti objavljena kasnije. Ovo nam pravi problem, jer mi možemo objaviti CRL listu u jednom trenutku i da se odmah u narednom trenutku desi novo povlačenje i onda ćemo mi objaviti novu CRL listu, pa onda za 5 minuta novo povlačenje – nova CRL lista itd. Šta je ovdje problem? Pa moguće je da oni koji preuzimaju CRL listu neće preuzimati novu do onog zakazanog termina izalaska nove CRL liste (npr. svaki dan u 10:00), a u pravoj CRL listi imamo neke nove povučene sertifikate, što može dovesti do problema. Sada se nameće da bi servisi mogli preuzimati CRL listu svaki minut, ali i u to slučaju nam ostaje tih 60 sekundi da se desi neki problem (da se iskoristi sertifikat koji je zapravo povučen). Onda bismo mogli svaki sekund, ali to je već besmisleno, opterećujemo procesor, zagušavamo mrežu, a u većini slučajeva ćemo preuzimati identičan fajl bez ikakvih izmjena (bez novih povučenih sertifikata). CRL liste sadrže još **listu opozvanih sertifikata (tj. njihove serijske brojeve) sa razlozima povlačenja i vremenom povlačenja** ovo vrijeme je jako bitno, jer ako se npr. neki sertifikat iskoristio u 10:00, a povučen je u 10:05 sve je okej, nema nikakvih problema. CRL liste sadrže i **ekstenzije (samo u verziji 2), identifikator algoritma koji je primjenjen za digitalno potpisivanje CRL liste**, kao i **digitalni potpis CRL liste**.

CA tijelo može da generiše i base i delta CRL liste, uglavnom nema delta, radi samo sa base CRL listama. Većina CA tijela ima samo base CRL liste koje izdaju svaka 24 sata. Ako CA tijelo izdaje i base i delta CRL liste neko ko provjerava da li je sertifikat povučen prvo treba

provjeriti u base CRL listi, a zatim u delta CRL listi. U ovom slučaju kada se izdaju oba tipa CRL liste, base se izdaje rjeđe npr. svkih 7 dana, dok se delta CRL lista, lista koja predstavlja samo razliku u odnosu na base CRL listu izdaje npr. svaka 24 sata. Kada dođe do nove objave base CRL liste svi povučeni sertifikati sa delta CRL liste se dopisuju na base CRL listu.

OCSP je protokol i on može da kontaktira ili CRL listu ili neko drugo skladište npr. bazu. Krajnji korisnik tj. klijent OCSP respondera mora da obezbijedi kopiju sertifikata OCSP respondera da bi mogao da verifikuje digitalno potpisan odgovor na upit, jer je možda odgovor potpisan nekim drugim privatnim ključem, a ne onim istim koji je u sertifikatu CA tijela. U ovim slučajevima digitalno potpisivanje ima negativan uticaj na performanse – na šta se misli? Na primjer moj sajt povuče zadnju izdatu CRL listu, provjeri jedan digitalni potpis na CRL listi i ima spisak svih povučenih sertifikata do tog trenutka, može da ih stavi npr. u mapu i ako mu u nekom trenutku dolazi 10 000 zahtjeva on vrlo brzo i lako u jednoj sekundi provjeri sve sertifikate da li povučeni. Šta se dešava kod OCSP-a? Ako nam dolazi 10 000 zahtjeva mi moramo OCSP-u da pošaljemo 10 000 zahtjeva, da dobijemo 10 000 odgovora i da validiramo 10 000 potpisa tih odgovora što je znatno sporije od prvog slučaja. OCSP nam daje 100% sigurnost da li je sertifikat povučen, dok CRL lista ne, ako je nismo upravo preuzeli. Problem kod OCSP-a je i taj što je izložen na mreži i podložan je DOS ili DDOS napadu. Rješenje je da imamo više OCSP respondera. Mjesto za dohvaćanje CRL liste takođe može biti pod napadom, ali za razliku od OCSP-a imamo barem neku „staru“ CRL listu pa nam može poslužiti u nekoj mjeri, dok OCSP ne može nikako ako je pod napadom.

Iz ovoga vidimo da ni CRL lista ni OCSP protokol nisu idelani, pa se u praksi vrši nekakva kombinacija oba pristupa.

Na CRL listi se mogu naći i suspendovani sertifikati, to su sertifikati koji mogu biti vraćeni ponovo u upotrebu. Rijetko se koristi, najčešće se koristi u kompanijama, kada neko ode na godišnji odmor onda mu se suspenduje sertifikat da nema pristup sistemu u tom vremenu.

14. Kako vjerovati CRL listi i kako zanti koji algoritam koristi?

(Gemini:) Povjerenje u CRL listu, slično povjerenju u sam sertifikat, ne dolazi automatski, već se zasniva na provjeri njene autentičnosti i aktuelnosti. Evo ključnih koraka i principa:

1. **Izdavač CRL liste:** CRL listu izdaje isti Sertifikacioni Autoritet (CA) koji je izdao sertifikate navedene u toj listi. Dakle, ako provjeravate sertifikat izdat od strane "Primjer CA", očekujete da CRL listu za taj sertifikat izda takođe "Primjer CA".
2. **Digitalni potpis CA:** Kako bi se osiguralo da CRL listu nije kreirao ili mijenjao neko neovlašten, CA digitalno potpisuje CRL listu prije nego što je objavi.
3. **Verifikacija potpisa na CRL listi:** Da biste vjerovali sadržaju CRL liste, morate verifikovati digitalni potpis na njoj. Ova verifikacija se vrši korištenjem javnog ključa CA koji je izdao CRL listu. Javni ključ tog CA se nalazi u njegovom CA sertifikatu.
4. **Povjerenje u CA sertifikat:** Da biste verifikovali potpis na CRL listi, morate vjerovati CA sertifikatu čiji javni ključ koristite za verifikaciju. Povjerenje u taj CA sertifikat se uspostavlja kroz lanac povjerenja (do pouzdanog root CA u vašem trust store-u) i provjerom validnosti samog CA sertifikata (da li je istekao, da li je i on opozvan, itd.).

5. **Aktuelnost CRL liste:** CRL liste imaju definisan period važenja (obično naznačen poljem *nextUpdate*). Važno je provjeriti da li je CRL lista koju koristite aktuelna i da nije istekla. Ako je istekla, potrebno je preuzeti noviju verziju.

Kada se govori o algoritmu koji "koristi" CRL lista, najčešće se misli na **algoritam koji je korišten za digitalno potpisivanje te CRL liste od strane CA**.

1. **Informacija je ugrađena u CRL:** Standardni format CRL liste (X.509 v2) sadrži polje koje eksplicitno navodi algoritam koji je korišten za potpisivanje liste. Ovo polje se obično naziva *signatureAlgorithm*.
2. **Sadržaj *signatureAlgorithm* polja:** Ovo polje obično kombinuje algoritam za heširanje (poput SHA-256, SHA-384, SHA-512) i algoritam za asimetrično potpisivanje (poput RSA, ECDSA). *Na primjer, vrednost može biti sha256WithRSAEncryption ili ecdsa-with-SHA384.*

(Prezentacija:) Sertifikaciono tijelo (CA tijelo) digitalno potpisuje CRL listu, tako da su korisnici PKI sistema sigurni u autentičnost i integritet informacija sadržanih u datoj listi.

(ChatGPT:) CRL listi se vjeruje zato što je digitalno potpisana od strane CA tijela koje je izdalo sertifikate (Povjerenje se bazira na tome da je potpisana od strane pouzdanog CA tijela (iz trust store-a)).

Algoritam se prepoznaje iz CRL fajla, u CRL fajlu se nalazi polje *signatureAlgorithm* (npr. *sha256WithRSAEncryption*) i taj algoritam se koristi za digitalno potpisivanje CRL liste.

a. Šta se sve nalazi u CRL listi?

Standard X.509 v2 definiše osnovni skup informacija koje treba da sadrži CRL lista:

- verzija formata CRL liste predstavlja oznaku strukture date liste koja je specificirana u standardu X.509 pri čemu su validne verzije 1 i 2
- naziv izdavača CRL liste
- vrijeme objavljivanja CRL liste
- vrijeme kada će CA tijelo objaviti sljedeću, ažuriranu CRL listu, nova verzija može biti objavljena i prije navedenog vremena, ali ne smije biti objavljena poslije navedenog vremena
- popis opozvanih sertifikata sa razlogom njihovog povlačenja – opozvani digitalni sertifikati su jednoznačno određeni njihovim serijskim brojem
- *ekstenzije – ovaj identifikator se koristi samo u verziji 2 i sadrži dodatna identifikaciona polja kao što su:*
 - *redni broj liste*
 - *distribucione tačke liste*
- identifikator algoritma koji je primjenjen za digitalno potpisivanje date CRL liste
- digitalni potpis CRL liste

b. Dodatni atribut koji ne treba biti u prethodnom pitanju?

Treba se skontati da ne ide tu.

c. Da li se mogu koristiti različiti algoritmi?

Valjda da :o

15. Šta je TimeStamp Authority?

Imamo digitalni sertifikat i sa njim potpišemo neki dokument danas i naš digitalni sertifikat se povuče sutra ili istekne i neko gleda taj naš potpisani dokument za 5 godina i šta on može zaključiti na osnovu samo toga – on može da uzme naš javni ključ i da validira da smo mi potpisali dokument i da dokument nije mijenjan, ali i da smo potpisali dokument nevalidnim sertifikatom, jer on ne može da vidi da je u trenutku potpisivanja dokumenta sertifikat bio validan. Šta nam sada treba? – treba nam da ugradimo vrijeme potpisivanja. E sada kako ćemo ugraditi vrijeme kada je potpisan dokument, ako ga budemo mi pisali možemo staviti šta god hoćemo, npr. neko vrijeme iz prošlosti kada je naš sertifikat bio validan, a danas npr. nije. Zato nam treba vremenski autoritet, tj. TSA – TimeStamp Authority, treća strana od povjerenja, koji će nam dati vremenski pečat koji će nam služiti za retroaktivno utvrđivanje da je digitalni sertifikat bio validan u trenutku potpisivanja.

TSA mora da koristi pouzdan izvor vremena, mora da koristi vrijeme generisano iz tog pouzdanog izvora, mora na neki način da poveže taj vremenski pečat i dokument na koji se on stavlja da se taj jedan generisani vremenski pečat ne bi koristio na x dokumenata (u ovom uslovu TSA ne mora da vidi dokument, ali mora nekako da ga poveže).

TSA može biti eksterna kompanija ili dio CA tijela.

a. Kako funkcioniše TimeStamp?

Onaj ko potpisuje dokument generiše hash tog dokumenta i šalje ga ka TSA i TSA generiše vremenski pečat u trenutku kada dobije hash.

TSA konkatenira dobijeni hash i generisani vremenski pečat i pravi hash vrijednost, koju zatim digitalno potpisuje svojim privatnim ključem.

TSA sada šalje digitalno potpisani hash i sam TimeStamp onom ko ga je zahtijevao.

Postavlja se pitanje zašto se šalje i sam TimeStamp? Da bi početna strana mogla ugraditi TimeStamp u dokument i da bi TimeStamp u budućnosti mogao biti validiran.

Validacija se radi tako što će neko ko bude radio validaciju prvo hash-irati sam dokument, pa onda napraviti hash konkateniranog hash-a dokumenta i TimeStamp-a, a zatim javnim ključem TSA dektipovati potpis i na kraju provjeriti da li se ta dva hash-a poklapaju.

Na ovaj način digitalno potpisan dokument nadživljava digitalni sertifikat.

b. Primjer u kom moraš prepoznati da treba TimeStamp i da onda objasniš?

TSA se može koristiti kod elektronskog glasanja.

Neki primjer sa ličnom kartom.

16. Kako razlikovati samopotpisani root sertifikat od običnog samopotpisanog sertifikata?

Oba mogu biti samopotpisana, ali samo root CA ima:

- Basic Constraints: CA:TRUE
- Issuer = Subject
- Key Usage – uključuje keyCertSign

Obični samopotpisani sertifikat može imati Issuer = Subject, ali:

- NEMA CA:TRUE
- NEMA pravo da potpisuje druge sertifikata (nije postavljeno keyCertSign)

a. Imaš root CA i običan CA, nema trust store, nema ništa drugo, kako razlikovati koji je root, a koji je običan CA?

Element	Root CA	Samopotpisani obični sertifikat
Issuer == Subject	DA	DA
Basic Constraints	CA:TRUE	CA:FALSE
Key Usage	Sadrži keyCertSign	Nema keyCertSign
Self-signed	Da	Da
Subject	Obično ime CA organizacije	Obično naziv servera ili korisnika